Computational Physics

Lecture 8: Random numbers

Sampling uniform distribution

- linear congruential generator and Mersenne Twister
- random seed
- probabilities and random events

Nonuniform random numbers

- exponential distribution
- Gaussian distribution

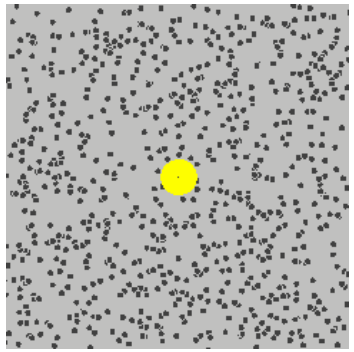
Reference: Chapter 10 of *Computational Physics* by Mark Newman

Random processes

Many physical processes involve in some way randomness:

Randomness can enter in modelling:

- **truly random problem**, such as radioactive decay, for which it is impossible to predict the time of the event
- **problems that** would not be random if we knew all the initial conditions, but look random and **can be effectively described as random**, such as the Brownian motion



Randomness can be important in tackling otherwise intractable problems:

- numerical integration in many dimensions
- sampling states in statistical mechanics with a huge conformational space
- find local minima in complex free energy landscapes with annealing

Numbers generated on a computer are usually not truly random, but a good generator produces numbers that reflect the desired properties of a random variable, hence it is called *pseudo-random number generator*.

Random numbers

First we need methods to generate random numbers. How to do it on a computer, which works deterministically?

Pseudo-random number generator: numbers generated on a computer are usually not truly random, but generated with a deterministic formula, producing numbers that reflect the desired properties of a random variable.

Historically, one of the simplest RNG is **linear congruential generator** (LCG), which you should never use!

It generates a sequence of pseudo-random numbers in accordance with an iterative procedure

$$x_{n+1} = (ax_n + c) \bmod m$$

for some parameters *a, m, c,* and a given starting point x_0.

The next number x_{n+1} in a sequence depends only on the present one x_n.

Random numbers

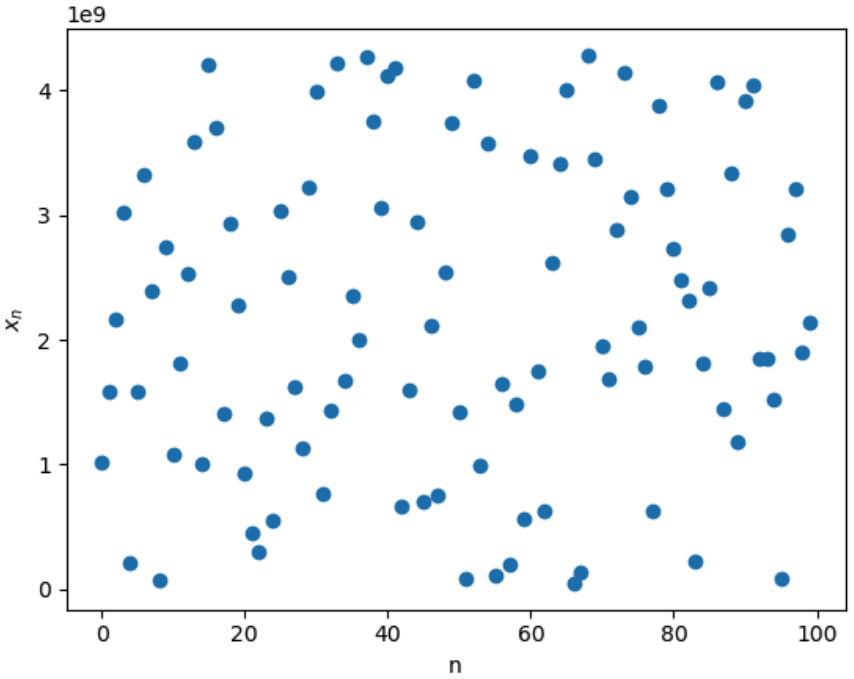
If you implement the LCG, and compute 100 random numbers starting from x_0=1, a = 1664525, c = 1013904223, m = 4294967296

```
N = 100
a = 1664525
c = 1013904223
m = 4294967296
x = 1
results = []

for i in range(N):
    x = (a*x+c)%m
    results.append(x)
plt.plot(results,"o")
plt.show()
```

The generated series of points looks pretty random. However:

- the series is not random, but deterministic. Knowing the constants and initial value, I can it reproduce exactly
- numbers fall in the following range: $0 \leq x_n < m$. Typically, one wants other ranges. The most common requirement is to be a real number r with $0 \leq r < 1$. This is obtained computing $r = x_n/m$
- the choice of *a, c, m* is important. If e.g. c and m are even, the process would generate only even or odd numbers.
- even with the same *a, c, m,* you can change the generated sequence by changing x_0, which is called the **seed**.



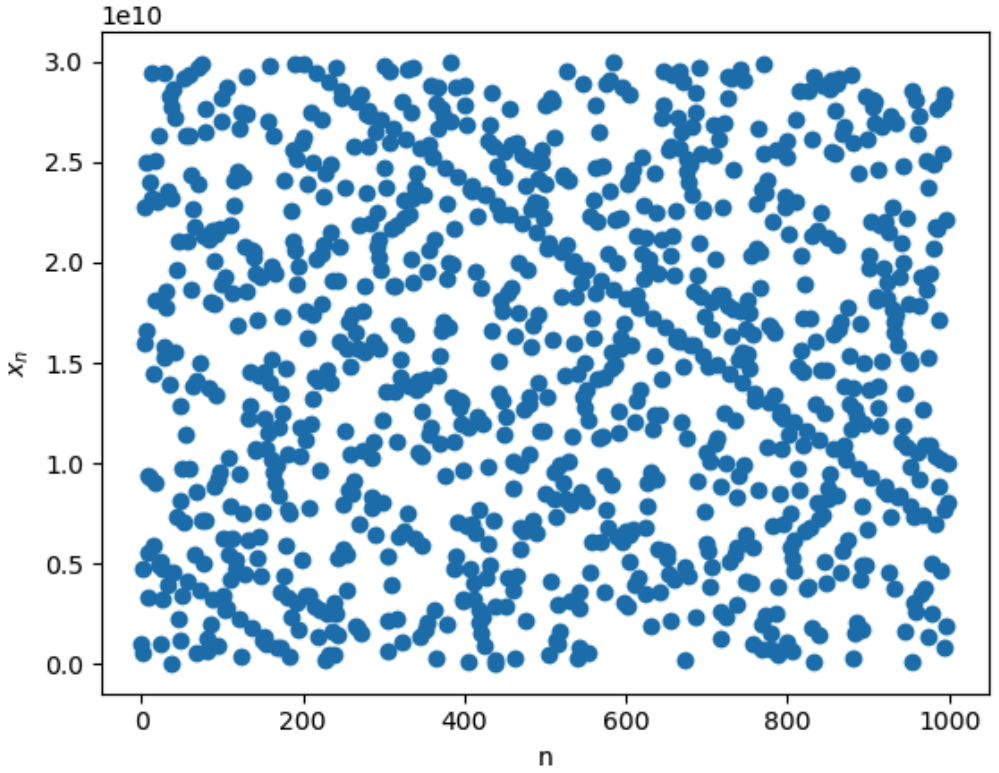
Random numbers

- it has a serious flaw: there are correlations between successive values, while we want complete uncorrelation (period at most *m*). Correlations can always create significant yet subtle errors in physics calculations

Just change m and increase N:

```
N = 1000
a = 1664525
c = 1013904223
m = 30000000000
x = 1
results = []

for i in range(N):
    x = (a*x+c)%m
    results.append(x)
plt.plot(results,"o")
plt.xlabel("n")
plt.ylabel("$x_n$")
plt.show()
```



Random numbers

LCG is not and should not be used in any serious calculations.

Other methods have been developed over the years.

The general method of choice is **Mersenne Twister** random number generator which is quite complex, but fortunately implemented by default in many programming environments and we will take it for granted.

MT has a long period of 2^{19937} - 1, passes most statistical randomness tests, is fast, and is suitable for physical applications.

In Python you can find it in the random package and also in numpy, e.g.:

```
import random
import numpy as np
random.random()  #random floating in [0,1)
random.randrange(m,n,k) #random integer from m to n-1 in steps of k
np.random.rand() #random floating in [0,1)
```

at every call, each function will generate a new random number extracted from the corresponding distribution

Random seed

Most random number generators generate a sequence iteratively starting from a specific starting point, which is specified by an input value, the seed. The seed determine the entire sequence.

Running the program from the same seed will always generate identical outcome.
This is important especially for debugging: a program error might occur after some time, and you need, aside from the same architecture, the same random sequence to reproduce the same error at the same time.

Remember for parallel production runs on a cluster to change the seed!!
E.g. you want multiple independent runs of some particle system involving random forces with same initial conformation, temperature, etc. Changing the seed will decorrelate the systems over time, and allow independent sampling.

Random seed

Without the seed Python will change it when being recalled from the start, making the behavior unpredictable. Calling the function will generate always a different sequence.

```
from random import random
for i in range(4):
    print(random())
```

```
0.7167761932243059
0.9504979611401653
0.032281765114187 12
0.4156131260160665
```

```
0.4004542367443189
0.38683965631244965
0.31610367551075425
0.33405065103792764
```

When specifying the seed, the called sequence will be always the same. Inputting the seed and then remembering to change it in production runs makes the physical simulations more reproducible

```
#random seed
from random import random,seed
seed(42)
for i in range(4):
    print(random())
```

```
0.6394267984578837
0.025010755222666936
0.27502931836911926
0.22321073814882275
```

```
0.6394267984578837
0.025010755222666936
0.27502931836911926
0.22321073814882275
```

Note that in some languages and program the standard behavior is instead having the same seed, so always pay attention.

Probabilities and random events

If we want an **event to happen with probability** p:

- extract a random number in $r \in [0,1)$ and check if $r < p$
- perform the action entailed by the event (e.g. a particle moving)

E.g. we have a biased coin that when tossed gives head 20% of the time and tail 80%.
We can reproduce a sequence of tossing in the following way:

```
from random import random
for i in range(5):
    if random()<0.2:
        print("H")
    else:
        print("T")
```

```
T
T
T
H
T
```

Example of decay of an isotope

Some physical processes are truly random, for instance **radioactive decay**

Take N(0)=1000 atoms of the radioisotope ^{208}Tl (thallium), which decays ^{208}Pb (lead) with half-life of τ = 3.053 mins. Simulate the decay, mimicking the randomness of the decay. On average:

$$N(t) = N(0)2^{-t/\tau}$$ # of remaining Tl atoms at t

$$N(0) - N(t) = N(0)(1 - 2^{-t/\tau})$$ # of decayed Tl atoms at t (or # of created Pb)

$$P(t) = \frac{N(0) - N(t)}{N(0)} = 1 - 2^{-t/\tau}$$ decaying probability of a single atom in *t* (fraction of particles at time t)

(1 - P(t) is the survival probability)

Start at t=0 with two counters N_Tl=1000 and N_Pb=0.
Divide the time into timesteps of 1s (<< τ), and at each timestep check that each of the remaining thallium atoms N_tl has decayed, drawing from the probability at t=1s

$$P(1) = 1 - 2^{-1/\tau}$$

All the decayed atoms are then subtracted from the counter N_tl and added to N_Pb.

Example of decay of an isotope

```
# Constants
NTl = 1000              # Number of thallium atoms
NPb = 0                 # Number of lead atoms
tau = 3.053*60          # Half life of thallium in seconds
h = 1.0                 # Size of time-step in seconds
p = 1 - 2**(-h/tau)     # Probability of decay in one step
t = 0                   # time
ts = 1                  # Timestep
tmax = 1000             # Total time

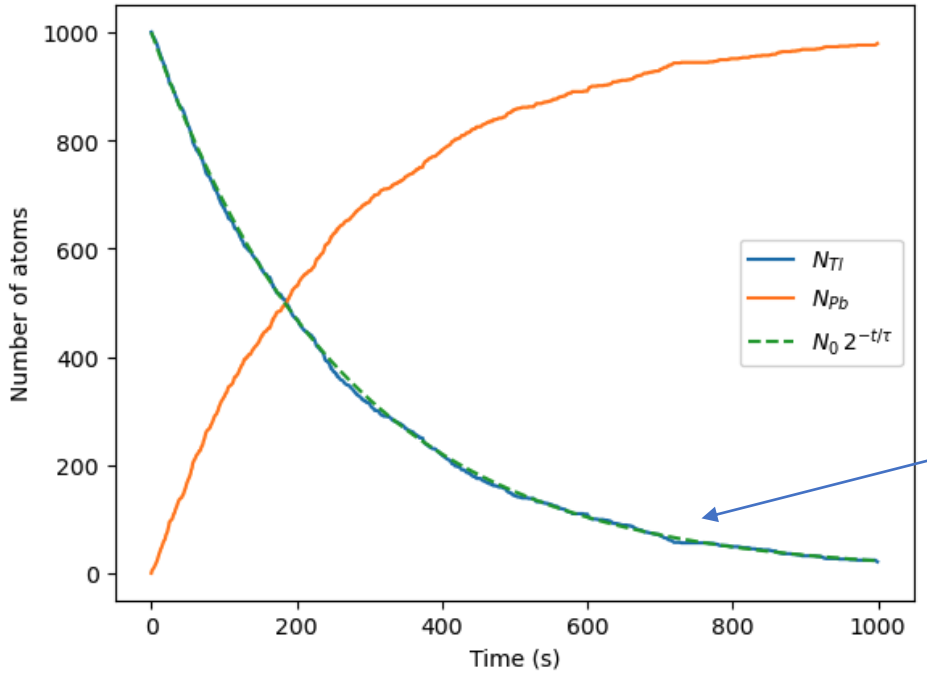
# time
list_t = []
list_NTl = []
list_NPb = []

# Main loop
for _ in range(0,int(tmax/ts)):
    list_t.append(t)
    list_NTl.append(NTl)
    list_NPb.append(NPb)

    # Calculate the number of atoms that decay
    decay = 0
    for i in range(NTl):
        if np.random.rand()<p:
            decay += 1
    NTl -= decay
    NPb += decay
    t += ts
```

```
# Make the graph
plt.plot(list_t,list_NTl,label="$N_{Tl}$")
plt.plot(list_t,list_NPb,label="$N_{Pb}$")
plt.xlabel("Time")
plt.ylabel("Number of atoms")
plt.legend()
plt.show()
```

Example of decay of an isotope

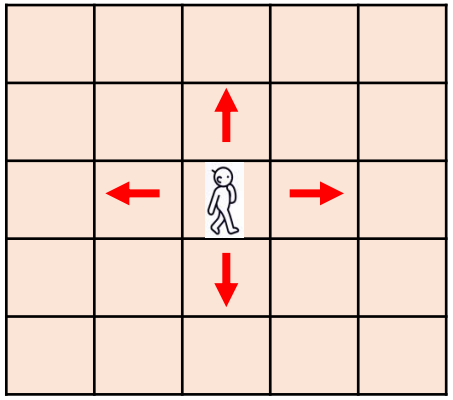


random wiggles, or noise, inherent to random processes

Average dashed curve fits well but is not perfect. Why? How to improve?

Example of Brownian motion on lattice

Brownian motion is the motion of a large particle immersed in a fluid of much smaller and lighter particles. Their interactions causes a seemingly random motion.



You can make a simple 2D computer simulation considering the large particle moving in a square lattice starting e.g. from the grid point x(0)=0,y(0)=0. At each simulation step, to represent the random collisions, choose a random direction--up, down, left, right--and move the particle in that direction (use 1 as lattice constant).
This is a **random walk** (it can be the movement of a drunken person).

The system is open, so just save the coordinates at each step. Try e.g. 1000 steps and look at the trajectory.

Bonus question: try performing multiple independent trajectories and look at e.g. x(t)-x(0).
Compute also <x(t)-x(0)> and $\langle (x(t)-x(0))^2 \rangle$, where the average is intended across independent trajectories, taking each average at fixed time. Plot then these quantities as a function of time, and look at the time-dependence, and comment on it.
The second quantity is known as **mean square displacement**, MSD.

Bonus question 2: You can also make the system enclosed in a box of size L (between –L/2, L/2): if the particle wants to move outside the border (e.g. $i \geq L/2$), just choose another direction randomly.

Example of Brownian motion on lattice

Single particle in an open system

```python
import numpy as np
import matplotlib.pyplot as plt

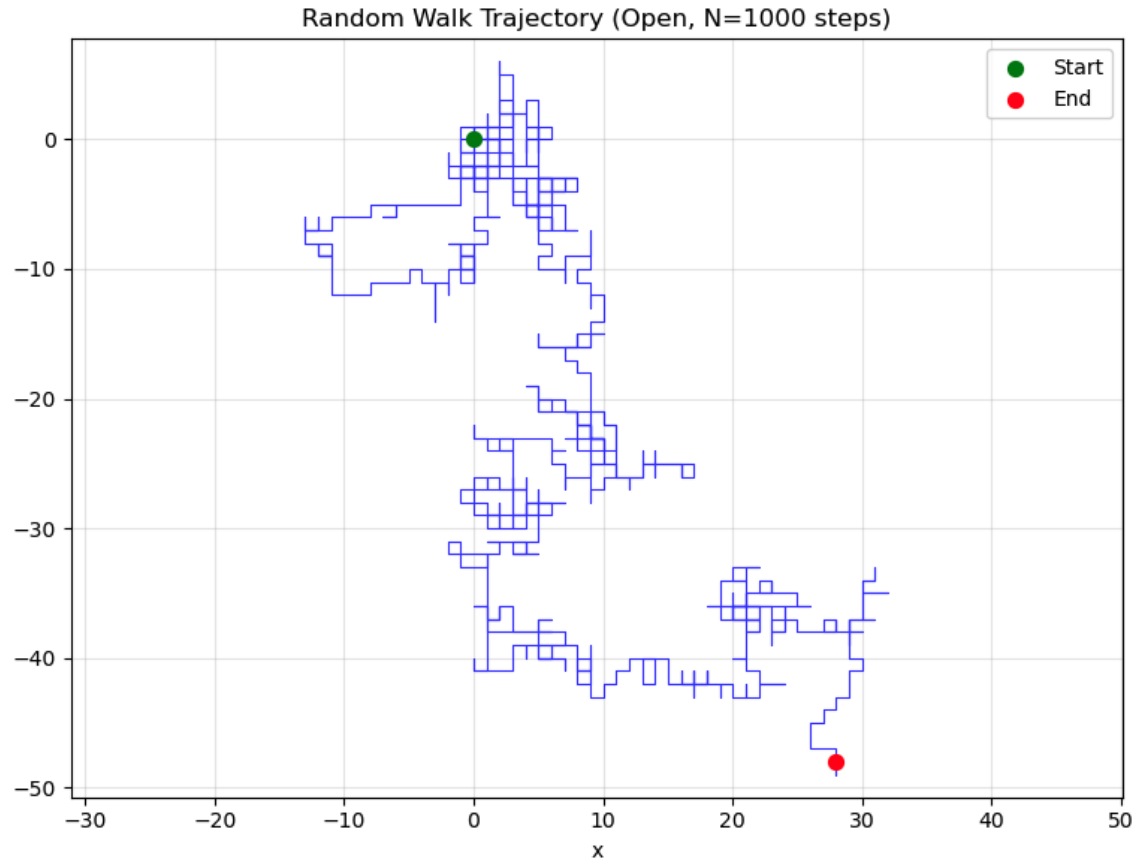
# Directions: right, left, up, down
dirs = np.array([[1, 0], [-1, 0], [0, 1], [0, -1]])

# ========== Single trajectory (open system) ==========
N = 1000
pos = np.array([0, 0])
x = [pos[0]]
y = [pos[1]]

for i in range(N):
    direction = np.random.randint(4)
    pos = pos + dirs[direction]
    x.append(pos[0])
    y.append(pos[1])

# Plot
plt.figure(figsize=(8, 6))
plt.plot(x, y, linewidth=0.8, color='blue', alpha=0.8)
plt.scatter([0], [0], color='green', s=50, zorder=5, label='Start')
plt.scatter([x[-1]], [y[-1]], color='red', s=50, zorder=5, label='End')
plt.xlabel('x')
plt.ylabel('y')
plt.title(f'Random Walk Trajectory (Open, N={N} steps)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.tight_layout()
plt.show()
```

Remember that in general it is a bad idea to initialize an array to save entire trajectory!! Best practice is to generate coordinates on the fly, and save trajectory only when you need



Example of Brownian motion on lattice

<x(t)-x(0)> and <(x(t)-x(0))^2>

```
# ========== Average and Mean Square Displacement ==========
N_stat = 5000
M = 500  # different realizations

# pos[0, :] = x of all realizations
# pos[1, :] = y of all realizations
pos = np.zeros((2, M))

sum_x = [np.mean(pos[0, :])]
sum_x2 = [np.mean(pos[0, :]**2)]

for i in range(N_stat):
    # make a move for all realizations
    for j in range(M):
        direction = np.random.randint(4)
        pos[:, j] = pos[:, j] + dirs[direction]

    # average and MSD
    sum_x.append(np.mean(pos[0, :]))
    sum_x2.append(np.mean(pos[0, :]**2))

mean_dx = np.array(sum_x)
msd_x = np.array(sum_x2)

t = np.arange(N_stat + 1)
theory = 0.5 * t
```

I am keeping the instantaneous coordinates at t, without saving all of them, and just computing averages on the fly

```
plt.figure(figsize=(10, 5))

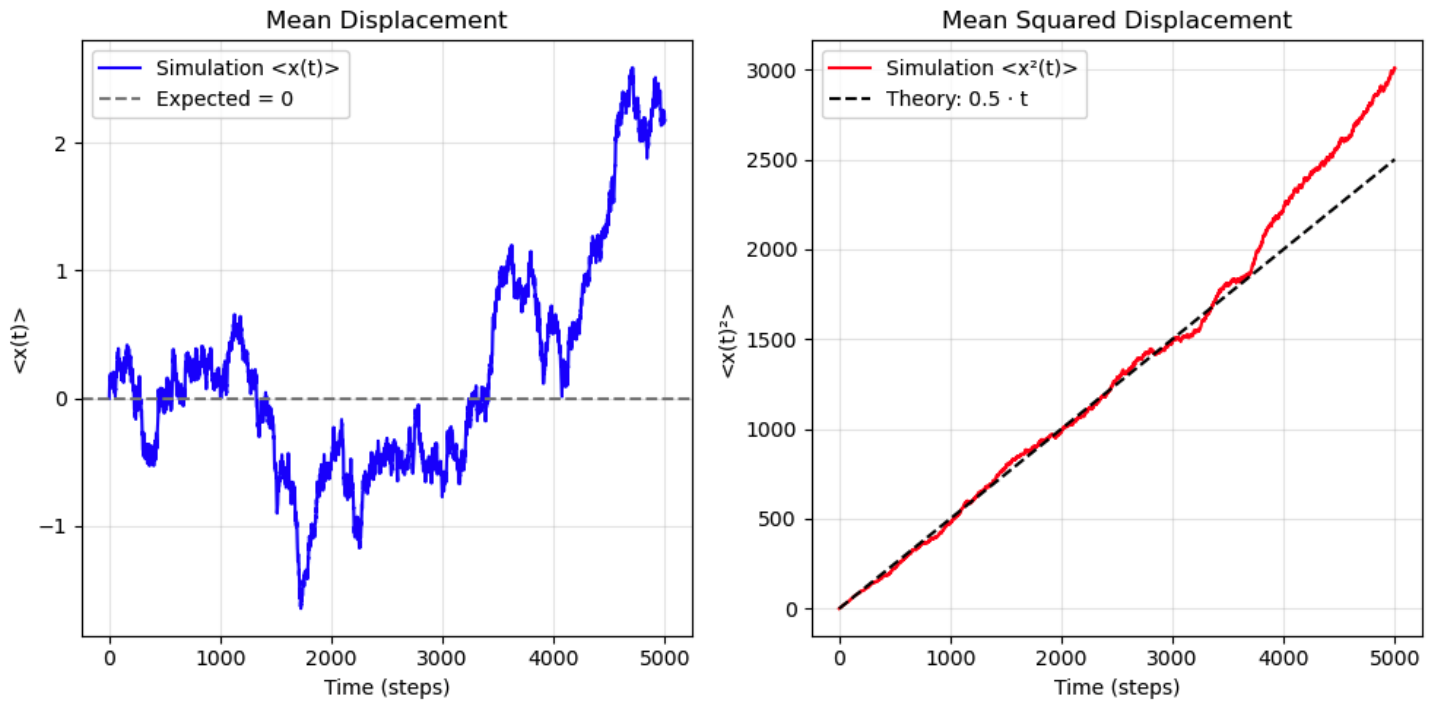
# Mean displacement
plt.subplot(1, 2, 1)
plt.plot(t, mean_dx, label='Simulation <x(t)>', color='blue')
plt.axhline(0, color='gray', linestyle='--', label='Expected = 0')
plt.xlabel('Time (steps)')
plt.ylabel('<x(t)>')
plt.title('Mean Displacement')
plt.legend()
plt.grid(True, alpha=0.3)

# Mean squared displacement
plt.subplot(1, 2, 2)
plt.plot(t, msd_x, label='Simulation <x²(t)>', color='red')
plt.plot(t, theory, label='Theory: 0.5 · t', color='black', linestyle='--')
plt.xlabel('Time (steps)')
plt.ylabel('<x(t)²>')
plt.title('Mean Squared Displacement')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Example of Brownian motion on lattice

<x(t)-x(0)> and $<(x(t)-x(0))^2>$



$\langle (x(t) - x(0))^2 \rangle = t / 2$. The typical signature of diffusion is having MSD $\propto t$. On average, the particle doesn't move, while the variance increases with time.

Increasing the independent realizations improves smoothness of averages

Nonuniformly distributed random numbers

In many cases we deal with random numbers that have a probability density distribution $p(x)$ which is non-uniform and in an interval which in general can be $(-\infty, +\infty)$, e.g. exponential, Gaussian, power-law distributions.

We can use the **inverse transformation sampling** for this purpose

Idea:

1) draw a number z using the uniform distribution $q(z) = \begin{cases} 1 \; if \; z \in [0,1) \\ 0 \;\; otherwise \end{cases}$

2) use some function $x = f(z)$ that connects $p(x)$ and $q(z)$ to transform z into a number x, such that x has the correct distribution $p(x)$

Nonuniformly distributed random numbers

How to find $x = f(z)$?

Suppose you have such function $x = f(z)$, which is deterministic, continuous and monotonic:
if z is a random number with distribution $q(z)$, also x will be a random one with distribution $p(x)$.

An infinitesimal interval is transformed as:

$$[z, z + dz] \rightarrow [x, x + dx] \text{ with } dx = |f'(z)|\, \mathrm{d}z$$

and thus the probability in that interval remains the same after transformation

$$\boxed{q(z)dz = p(x)dx}$$

Let's integrate on but sides up to z and $x = f(z)$

$$\int_{-\infty}^{z} q(z')dz' = \int_{-\infty}^{x} p(x')dx' \longrightarrow \int_{0}^{z} dz' = \int_{-\infty}^{x} p(x')dx' \longrightarrow z = \int_{-\infty}^{x} p(x')dx'$$

$$q(z) = \begin{cases} 1 \; if \; z \in [0,1) \\ 0 \;\; otherwise \end{cases}$$

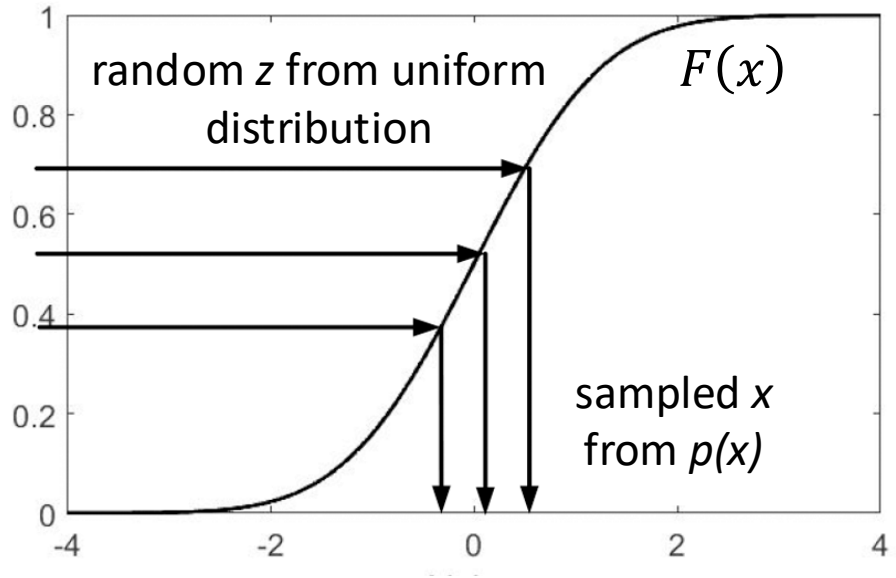
Nonuniformly distributed random numbers

If we can integrate $z = \int_{-\infty}^{x} p(x')dx' = F(x)$ and invert it, we end up with $x = f(z) = F^{-1}(z)$

Once we obtained $x = f(z)$, if we draw a random uniform number z in [0,1), and apply the equation, we end up with a number x as it was extracted from $p(x)$

$$F(x) = \int_{-\infty}^{x} p(x')dx'$$

is called **cumulative distribution**: it is the probability of having the extracted number to be less than the value *x*.



Challenges: Sometimes, evaluating (*x*) and/or $F^{-1}(z)$ explicitly is difficult. In such cases, numerical integration and/or non-linear equation solvers may be required.

Inverse transform sampling: Exponential distribution

Exponential Distribution $p(x) = \mu e^{-\mu x}$ defined in the interval $[0, +\infty)$

The cumulative distribution function:

$$F(x) = \int_{-\infty}^{x} p(x')dx' = \int_{0}^{x} \mu e^{-\mu x'} dx' = 1 - e^{-\mu x}$$

To apply inverse transform sampling, invert the equation $1 - e^{-\mu x} = z$

Solving for *x*, we obtain: $x = -\frac{1}{\mu}\ln(1 - z)$

If we extract a series of uniform random numbers z, *x* will have the exponential distribution (check yourself!!)

Example of decay of an isotope

Let's go back to the example of **radioactive decay**

$$P(t) = \frac{N(0) - N(t)}{N(0)} = 1 - 2^{-t/\tau}$$ is actually the cumulative probability of a particle decaying in [0,*t*].

You can write the probability density of decaying times in the following:

the probability of decaying in a small interval is $$P(dt) = 1 - 2^{-dt/\tau} = 1 - e^{-\frac{dt}{\tau}\ln 2} = \frac{\ln 2}{\tau} dt$$

The probability of decaying in [*t,t+dt*] is given by the probability of survival up to *t* (which is *1-P(t)*) times the decaying probability *P(dt)*

$$p(t)dt = 2^{-t/\tau}\frac{\ln 2}{\tau} dt$$

hence we arrive at a density distribution of times which is exponential.

Now, instead of doing the previous convoluted simulation of decaying, you can just extract 1000 random decaying times for each particle from the distribution, and then to compute e.g. Npb(t) just count how many particles have a decayed up to t.

Example of decay of an isotope

```python
import numpy as np
import matplotlib.pyplot as plt

# Constants
N = 1000                  # Number of atoms
tau = 3.053 * 60          # Half-life of thallium in seconds
tmax = 1000               # Total time (s)

# --- 1. Direct sampling of decay times from the exponential ---
# Inverse transform: t = - (tau / ln2) * ln(1-u)
u = np.random.rand(N)
decay_times = - (tau / np.log(2)) * np.log(1 - u)

# Sort them so we can easily count how many have decayed by time t
decay_times_sorted = np.sort(decay_times)

# --- 2. Reconstruct the counting curves from the sampled times ---
# Time grid
t = np.arange(0, tmax + 1, 1.0)

# Number of atoms that have decayed by time t (and those still alive)
N_decayed = np.searchsorted(decay_times_sorted, t, side='right')
N_Tl = N - N_decayed
N_Pb = N_decayed

# Theoretical curve
N_theory = N * 2**(-t / tau)
```

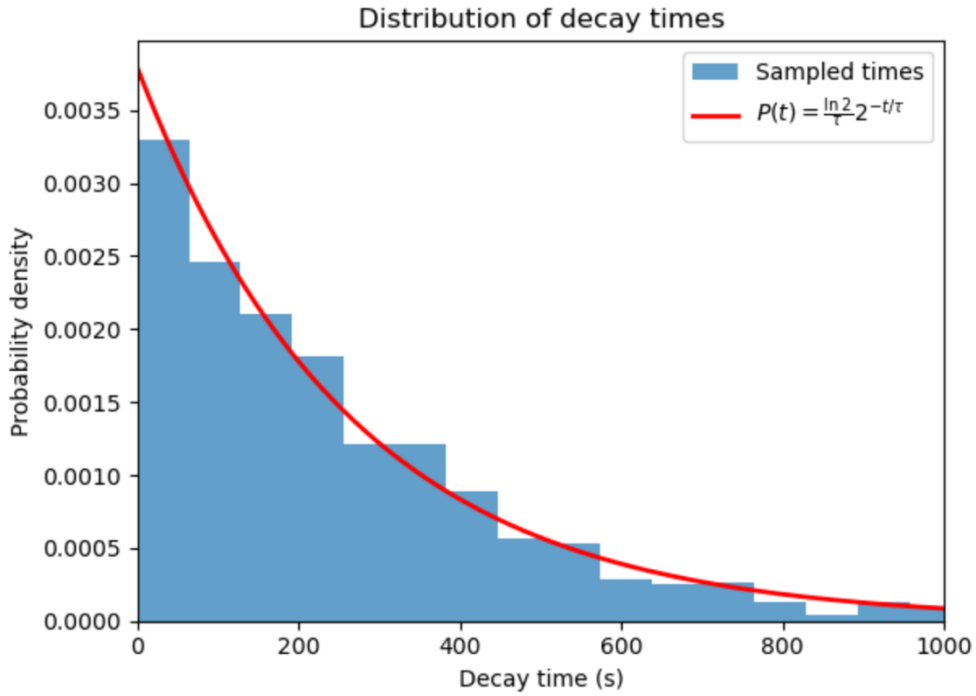
```python
# --- Plots ---
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

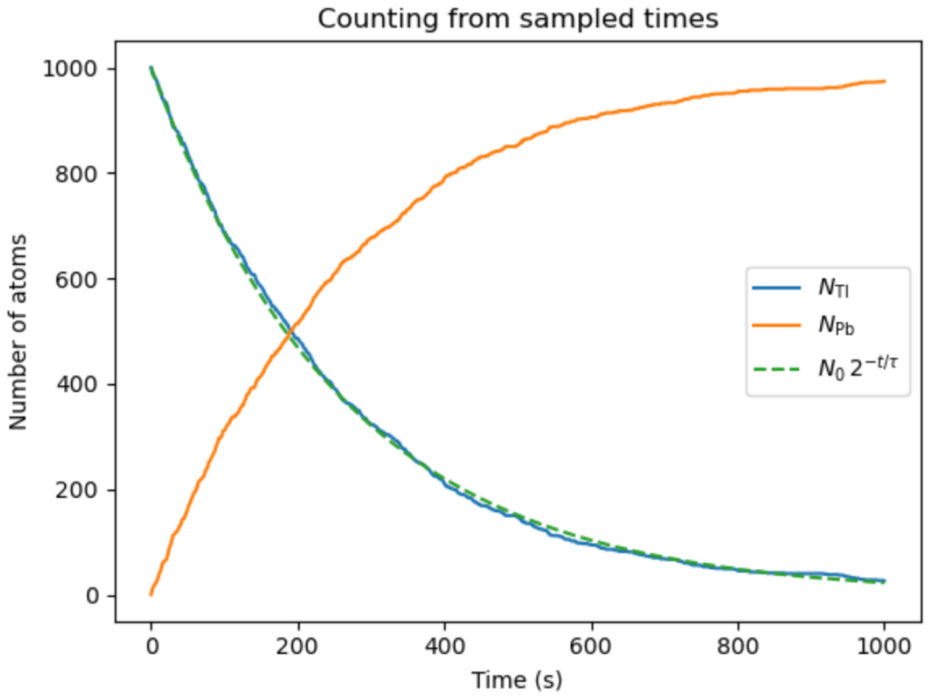
# Left: histogram of the decay times (the density)
axes[0].hist(decay_times, bins=40, density=True, alpha=0.7, label="Sampled times")
# Overlay the theoretical PDF
t_pdf = np.linspace(0, tmax, 500)
pdf = (np.log(2) / tau) * 2**(-t_pdf / tau)
axes[0].plot(t_pdf, pdf, 'r-', lw=2, label=r"$P(t)=\frac{\ln 2}{\tau}2^{-t/\tau}$")
axes[0].set_xlabel("Decay time (s)")
axes[0].set_ylabel("Probability density")
axes[0].set_title("Distribution of decay times")
axes[0].legend()
axes[0].set_xlim(0, tmax)

# Right: counting curves (same information as the original simulation)
axes[1].plot(t, N_Tl, label=r"$N_{\rm Tl}$")
axes[1].plot(t, N_Pb, label=r"$N_{\rm Pb}$")
axes[1].plot(t, N_theory, "--", label=r"$N_0\,2^{-t/\tau}$")
axes[1].set_xlabel("Time (s)")
axes[1].set_ylabel("Number of atoms")
axes[1].set_title("Counting from sampled times")
axes[1].legend()

plt.tight_layout()
plt.show()
```

Example of decay of an isotope





Inverse transform sampling: Gaussian distribution

Gaussian Distribution $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$

we need again to compute the cumulative distribution and invert the equation:

$$\int_{-\infty}^{x} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}} dx = z$$

No closed-form expression is known. However, one can resort to the **Box-Muller method**. Take two random numbers, *x* and *y*, both extracted from a Gaussian distribution with deviation σ. The probability that the vector *(x,y)* falls in some element $dxdy$ is

$$p(x)p(y)dx\,dy = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} dx\,dy = \frac{1}{2\pi\sigma^2} e^{-\frac{r^2}{2\sigma^2}} r dr d\theta = \frac{r}{\sigma^2} e^{-\frac{r^2}{2\sigma^2}} dr \frac{d\theta}{2\pi} = p(r)dr\; p(\theta)d\theta$$

in polar coordinates

properly normalized distributions

Inverse transform sampling: Gaussian distribution

If we generate a random value of r and θ from $p(r)$ and $p(\theta)$, and convert them to Cartesian coordinates we can retrieve x and y which are now Gaussian

To generate θ, consider $p(\theta) = \frac{1}{2\pi}$ is uniform with $\theta \in [0,2\pi)$. So extract a uniform number in [0,1) and multiply by 2π

To generate r, consider $p(r) = \frac{r}{\sigma^2} e^{-\frac{r^2}{2\sigma^2}}$ with $r \in [0, +\infty)$ and compute cumulative:

$$\frac{1}{\sigma^2} \int_0^r e^{-\frac{r^2}{2\sigma^2}} r dr = 1 - e^{-\frac{r^2}{2\sigma^2}} = z$$

Rearranging

$$r = \sqrt{-2\sigma^2 \ln(1-z)}$$

We have two Gaussian random numbers

$$x = r\cos\theta$$

$$y = r\sin\theta$$

Example of Gaussian distribution via Box-Muller method

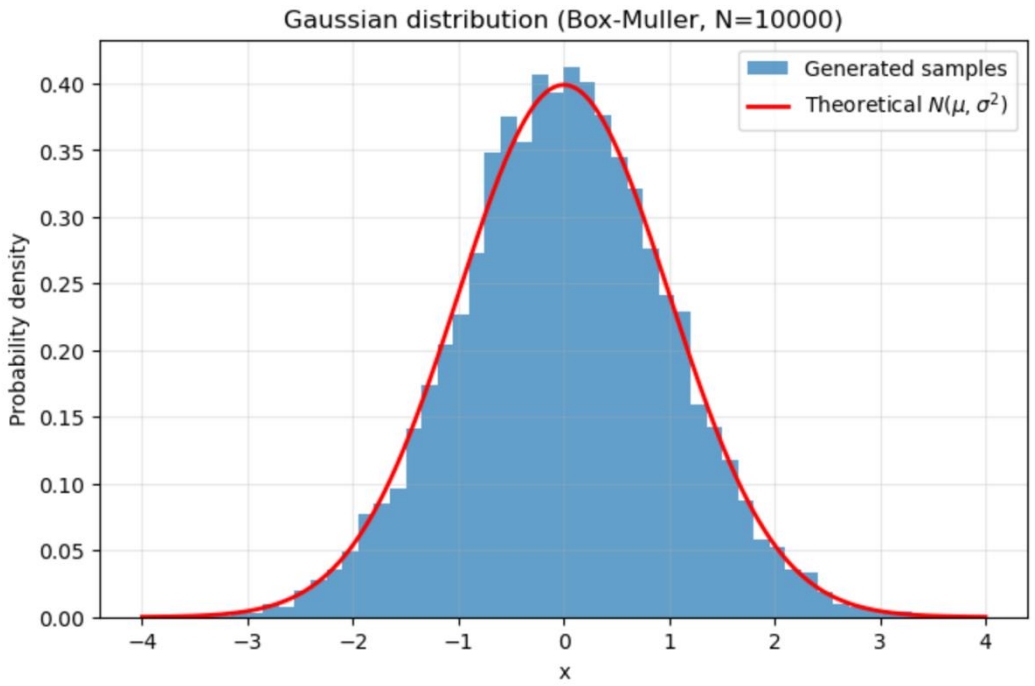
```python
# Parameters
sigma = 1.0          # standard deviation
mu = 0.0             # mean
N = 10000            # number of samples

def gaussian(sigma=1.0):
    """Box-Muller: returns two independent standard Gaussians,
    then scaled by sigma."""
    r = np.sqrt(-2 * sigma**2 * np.log(1 - np.random.rand()))
    theta = 2 * np.pi * np.random.rand()
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    return x, y

# Generate samples
samples = []
for _ in range(N // 2):
    x, y = gaussian(sigma)
    samples.append(x)
    samples.append(y)
samples = np.array(samples) + mu   # shift by mean if desired

# Theoretical PDF
x_theory = np.linspace(-4*sigma, 4*sigma, 500)
pdf = (1 / (sigma * np.sqrt(2*np.pi))) * np.exp(-0.5 * ((x_theory - mu)/sigma)**2)

# Plot
plt.figure(figsize=(8, 5))
plt.hist(samples, bins=50, density=True, alpha=0.7, label="Generated samples")
plt.plot(x_theory, pdf, 'r-', lw=2, label=r"Theoretical $N(\mu,\sigma^2)$")
plt.xlabel("x")
plt.ylabel("Probability density")
plt.title(f"Gaussian distribution (Box-Muller, N={N})")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



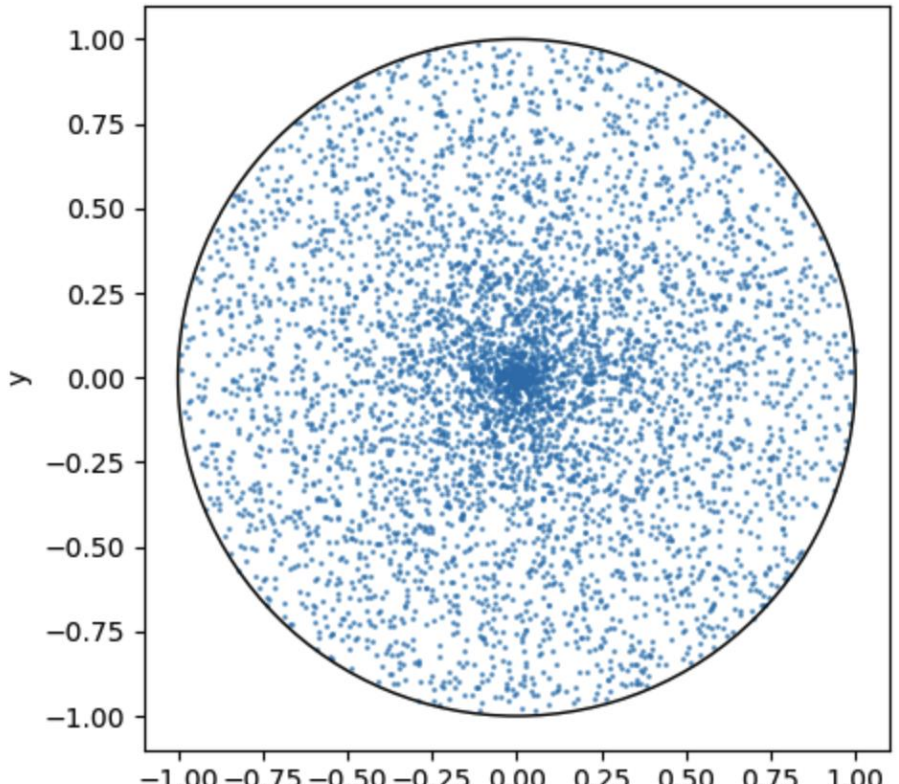
Inverse transform sampling: sampling points in a circle

Sample random points (x,y) inside a unit circle, so $x^2 + y^2 \leq 1$:

Expressing in polar coordinates $x = r\cos\theta$ $y = r\sin\theta$

naively one could think of sampling *r* and θ from two random distributions with $r \in [0,1)$ and $\theta \in [0,2\pi)$

But this gives the wrong result (points more clumped in the center)!



Inverse transform sampling: sampling points in a circle

Our distribution is uniform in x and y ($p(x) = 1$ and $p(y) = 1$) but not $p(r)$:

$$p(x)p(y)dx\,dy = dx\,dy = rdrd\theta = p(r)dr\,p(\theta)d\theta$$

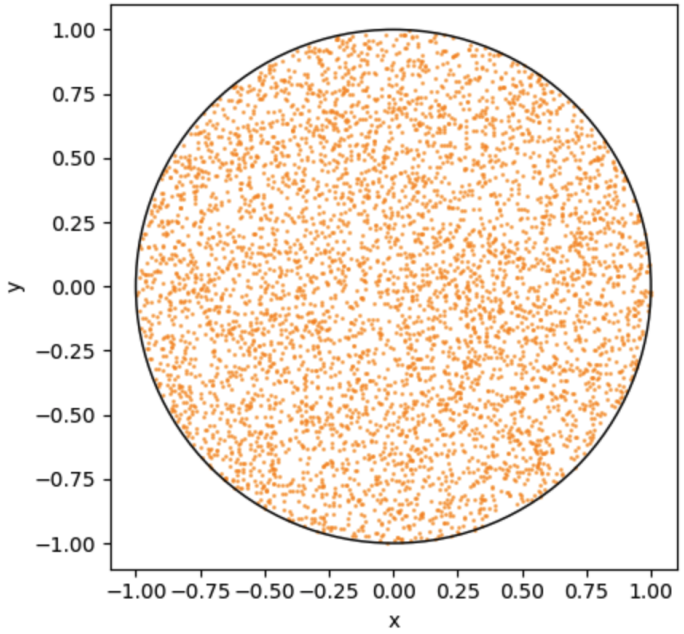
with

$$p(\theta) = \frac{1}{2\pi}$$

$$p(r) = 2r$$

$p(r)$ is not homogeneous, and we need to do this inverse transform

$$\int_0^r 2rdr = r^2 = z \longrightarrow r = \sqrt{z}$$



random points count grows linearly

Inverse transform sampling: sampling of an Isotropic Direction in 3D

One common problem in **Monte Carlo simulations** is the **random sampling of an isotropic direction in 3D space**. This issue arises in various contexts, such as:

- Sampling a random orientation of an **axially symmetric object** (e.g., a rod).

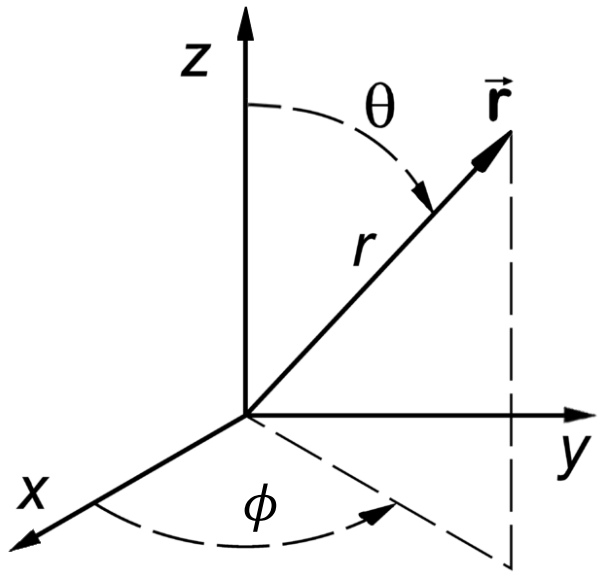
- Sampling the **momentum direction** of a particle.

This problem is equivalent to **choosing a random point on a unit sphere**. The coordinates x, y, z on the sphere can be parametrized using **azimuthal** and **polar angles**:

- $\phi \in [0, 2\pi)$ (azimuthal angle)
- $\theta \in [0, \pi)$ (polar angle)

Using these angles, the Cartesian coordinates are given by:

$$x = \sin\theta \cos\phi$$
$$y = \sin\theta \sin\phi$$
$$z = \cos\theta$$



Again, would be wrong choice to sample uniformly both angles

Inverse transform sampling: sampling of an Isotropic Direction in 3D

Looking at the solid angle: $d\Omega = \sin(\theta)\, d\theta d\phi$

we have $p(\theta) = \frac{1}{2}\sin(\theta)$ while $p(\phi) = \frac{1}{2\pi}$ is uniform (independent of each other)

$$\int_0^{\theta} \frac{1}{2}\sin(\theta)\, d\theta = \frac{1-\cos\theta}{2} = z \quad \longrightarrow \quad \theta = \arccos(2z-1)$$

```python
def sample_xyz_isotropic():
    """Sample a random direction uniformly on the unit sphere."""
    phi = 2 * np.pi * np.random.rand()
    costh = 2 * np.random.rand() - 1          # cosθ uniform in [-1, 1]
    sinth = np.sqrt(1 - costh**2)
    return sinth * np.cos(phi), sinth * np.sin(phi), costh

# Generate N isotropic points
N = 500
xplot, yplot, zplot = [], [], []

for _ in range(N):
    x, y, z = sample_xyz_isotropic()
    xplot.append(x)
    yplot.append(y)
    zplot.append(z)

# 3D scatter plot
fig = plt.figure(figsize=(7, 6))
ax = fig.add_subplot(projection='3d')

ax.scatter(xplot, yplot, zplot, s=10, alpha=0.7)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_title('Isotropic sampling on the unit sphere')
ax.set_box_aspect([1, 1, 1])   # equal aspect ratio
```

